Optimal Play in *Cookie Clicker*:
Purchase Order, Golden-Cookie Banking, Heavenly-Chip Allocation, and Ascension Timing as an Optimal Stopping Problem

Claude (Anthropic)
Prepared for the Cookie Monster+ project

October 25, 2026

Abstract

We model the idle game *Cookie Clicker* as a continuous-time growth process with discrete purchases and periodic resets ("ascensions") that trade the current run for a permanent production multiplier. Within explicit models we prove four results. (i) A pairwise exchange criterion orders purchases by $p/c + p/\Delta$, and the community "payback period" heuristic implements it exactly whenever no candidate is affordable. (ii) Holding cookies for the "Lucky!" golden cookie beats buying an item exactly when the item's payback time, measured on its expected income gain with clicks and golden-cookie effects included, exceeds $1/(0.15\, g\, \lambda_L)$, about two hours at base golden-cookie frequency. (iii) Heavenly chips spent on multiplicative upgrades are allocated optimally by sorting on $(1-1/m)/p$, a Smith-type ratio rule. (iv) In the late game, a telescoping lower bound on a Bellman equation yields a closed-form optimal ascension schedule: each run should multiply all-time cookies by the root $v^\star$ of $\frac{\beta}{3}(1-\frac{1}{v}) = (1-v^{-\gamma/3})/\gamma$, $\gamma = 3/\beta - 1$, where β is the run's growth exponent. This corrects the marginal value theorem by up to a factor of six. Because the closed form breaks down early in the game, we give an exact dynamic program over future ascensions that includes heavenly upgrades. In a simulator built from the game's own building formulas, the stopping rule and the planner reach on average 98.6% and 97.6% of the best outcome in hindsight in late-game scenarios, against 92% for the best fixed rule tuned per scenario. From a fresh start the planner beats every fixed rule tested by 56% after 30 days. All policies are implemented in the Cookie Monster+ mod.

1 Introduction

Cookie Clicker (Orteil, 2013–) is an incremental game. Cookies are produced by clicking and by buildings whose prices grow geometrically (each copy costs 1.15 times the previous one); upgrades multiply production; randomly appearing *golden cookies* grant temporary bonuses; and the player may at any time *ascend*, abandoning the current run in exchange for *prestige*, a permanent production multiplier that grows like the cube root of all cookies ever baked. Prestige also yields *heavenly chips*, a currency for permanent upgrades.

Four decisions recur throughout play: what to buy next, whether to keep cookies in the bank for golden-cookie rewards, how to spend heavenly chips, and when to ascend. This paper formalises each one, proves optimality results under stated assumptions, and validates the resulting policies numerically.

Contributions.

1. An exchange criterion for purchase order (Theorem 1), showing that the widely used "payback period" ranks purchases optimally whenever nothing is affordable (Corollary 2).
2. A marginal-return rule for holding a "Lucky!" bank (Proposition 4) and a closed form for the mean golden-cookie interval (Lemma 5).
3. An optimal ratio rule $(1 - 1/m)/p$ for spending heavenly chips on multiplicative upgrades (Theorem 6).
4. A time-rescaling lemma (Lemma 7) that reduces ascension timing to an optimal stopping problem, a tight lower bound giving the late-game optimum in closed form (Theorem 8), a proof that early resets are useless when prestige barely affects speed (Proposition 10), and an exact dynamic program for the general case (Proposition 11).
5. Rules for the rest of the game (Section 7): the value of an achievement through milk and, under time rescaling, in seconds of play (Proposition 13); Godzamok round trips, whose buff grows without limit when buildings are sold and bought back every frame (Proposition 14); the wrinkler factor $W(n) = 1 - s + snr$; why wrath cookies beat golden ones under automation, and why calm wins once clicks dominate (Propositions 16 and 17); and rules for popping wrinklers, the Grimoire (cast at full magic), the Pantheon, Garden breeding (Lemma 19), Stock Market loans, the dragon, seasons, sugar lumps, permanent slots and the Born again challenge run.
6. Numerical validation on a simulator using the game's building curves (Section 8).

Scope. "Optimal" always means optimal *within the stated model*. The proofs are exact under their assumptions; how well the assumptions fit the real game is tested numerically, not proved. The policies are implemented in Cookie Monster+, a fork of the Cookie Monster mod (Section 9).

2 Model

2.1 Mechanics used

A run starts with no buildings. Building k with n copies costs $p_k(n) = p_k(0)\,1.15^n$. Let A be the number of cookies baked in all previous runs and C the number baked in the current one. Ascending converts the run into prestige

$$P(A + C) = \big((A + C)/10^{12}\big)^{1/3}, \tag{1}$$

and every prestige level adds h percent to production, where the *prestige power* h becomes 1 once the player owns the heavenly upgrade *Legacy* (one chip) and buys five cheap store upgrades (*Heavenly chip secret* through *Heavenly key*, worth $5\% + 20\% + 25\% + 25\% + 25\%$). The production multiplier of a run that starts at prestige P is therefore

$$M(P) = 1 + 0.01\,h\,\lfloor P \rfloor. \tag{2}$$

Over the whole game the player receives $\lfloor P \rfloor$ heavenly chips, spent on permanent heavenly upgrades. Each heavenly upgrade has a price in chips and a set of parent upgrades, all of which must be owned first.

2.2 Notation

Symbol	Meaning
c	income rate (cookies per second)
b	cookies in the bank
$p_i,\ \Delta_i$	price of purchase i and the income it adds
$A,\ C$	cookies baked before the current run, and in it
P, h	prestige (1), prestige power
S	speed of a run: the factor by which permanent bonuses multiply all income
$F(x)$	run curve: cookies baked after x units of speed-one time
β	local growth exponent of a run, $\beta = cT/C$
τ	time lost per ascension

2.3 Assumptions

Assumption A1. Between purchases the income rate is constant, and purchases are instantaneous.

Assumption A2. The income added by a purchase does not depend on the order in which a given pair of purchases is made.

Assumption A3. Permanent bonuses multiply every income source (buildings, clicks, golden-cookie rewards) by the same factor S.

Assumption A4. Purchase decisions depend on the bank and holdings only through comparisons that are invariant under multiplying all incomes by a constant.

Assumption A5. (Section 6.3 only.) The run curve is a power law, $F(x) = \alpha x^{\beta}$ with $\beta > 1$.

3 What to buy next

Theorem 1 (Pairwise exchange)**.** *Assume A1–A2. Two purchases A and B are to be made consecutively, starting from a bank $b \le \min(p_A, p_B)$ and income c. Buying A first completes both purchases no later than buying B first if and only if*

$$E_A \le E_B, \qquad E_i \coloneqq \frac{p_i}{c} + \frac{p_i}{\Delta_i}. \tag{3}$$

Both orders end in the same state.

Proof. With A first, the bank reaches p_A after $(p_A - b)/c$ seconds and is then empty, so B follows after a further $p_B/(c + \Delta_A)$ seconds:

$$T_{AB} = \frac{p_A - b}{c} + \frac{p_B}{c + \Delta_A}, \qquad T_{BA} = \frac{p_B - b}{c} + \frac{p_A}{c + \Delta_B}.$$

The bank b cancels in $T_{AB} - T_{BA}$, and

$$T_{AB} \le T_{BA} \iff p_A\Big(\frac{1}{c} - \frac{1}{c + \Delta_B}\Big) \le p_B\Big(\frac{1}{c} - \frac{1}{c + \Delta_A}\Big) \iff \frac{p_A \Delta_B}{c + \Delta_B} \le \frac{p_B \Delta_A}{c + \Delta_A}.$$

Multiplying by $(c+\Delta_A)(c+\Delta_B)/(c\,\Delta_A\Delta_B) > 0$ gives $p_A/\Delta_A + p_A/c \le p_B/\Delta_B + p_B/c$. After both purchases the holdings, the empty bank and the income $c + \Delta_A + \Delta_B$ (by A2) coincide. □

Remark 1. Because both orders end in the same state, any objective of the form "reach a given state as early as possible" prefers the order with the smaller index. Buying, at each decision, the candidate with the smallest E_i at the current income is therefore *locally optimal*: no swap of two consecutive purchases improves it. A single global sort is impossible, because E_i depends on c, which grows.

Corollary 2 (Payback period)**.** *Cookie Monster ranks purchases by the payback period* $\mathrm{PP}_i = \max(p_i - b, 0)/c + p_i/\Delta_i$*. When no candidate is affordable,* $\mathrm{PP}_i = E_i - b/c$*, so the payback period and the exchange index induce the same ranking.*

Proposition 3 (Affordable pairs)**.** *If* $\max(p_A, p_B) \le b < p_A + p_B$*, buying A first is better if and only if* $\Delta_A > \Delta_B$*.*

Proof. Buying A immediately leaves $b - p_A < p_B$, and the remaining $p_A + p_B - b$ cookies arrive at rate $c + \Delta_A$: $T_{AB} = (p_A + p_B - b)/(c + \Delta_A)$. The expression for T_{BA} is symmetric, and the numerators agree. □

With a large bank the exchange index is therefore not exact. Section 8 shows the effect is small: over nine scenarios, replacing the payback period by the bank-independent index, or adding a look-ahead to tier upgrades, changes the time to reach a target by at most 0.35% in continuous play and 1.7% when purchases are made only once an hour.

4 Golden cookies and the bank

4.1 When to hold a "Lucky!" bank

A "Lucky!" golden cookie pays $\min(0.15\,b,\ 900\,c)\,g + 13$ cookies, where g collects golden-cookie gain multipliers. Below the cap ($b < 6000\,c$) the reward is linear in the bank.

Proposition 4 (Marginal value of banked cookies)**.** *Let* "Lucky!" *events occur at rate* λ_L *and let* $b < 6000\,c$*. A marginal banked cookie earns* $0.15\,g\,\lambda_L$ *cookies per second in expectation, while spending it on item* i *earns* $\bar{\Delta}_i/p_i$*, where* $\bar{\Delta}_i$ *is the expected increase in total income the item brings (Definition 1). Spending is better if and only if*

$$\frac{p_i}{\bar{\Delta}_i} \;<\; \theta := \frac{1}{0.15\,g\,\lambda_L}.$$

Proof. Below the cap, each "Lucky!" event pays $0.15\,g$ per banked cookie, and these events arrive at rate λ_L. By linearity of expectation the banked cookie returns $0.15\,g\,\lambda_L$ per second. A cookie spent on item i returns $\bar{\Delta}_i/p_i$ per second. Comparing the two marginal returns gives the claim. □

Without golden cookies or clicking, $\bar{\Delta}_i$ is the CpS gain Δ_i. With both automated, using Δ_i instead undervalues every purchase: with both frequency upgrades and a fast auto-clicker, golden cookies and clicks multiply a CpS gain about eightfold (Example 1), so purchases whose payback lies between θ and several times θ would be held although buying them is better.

The rate is $\lambda_L = \pi_L/\mathbb{E}[I]$, where $\pi_L \approx 0.42$ is the probability that a golden cookie is "Lucky!" and I is the interval between golden cookies.

Lemma 5 (Golden-cookie interval)**.** *Golden cookies spawn between* $t_{\min}$ *and* $t_{\max}$*. In each frame of the window the spawn probability is* $((t - t_{\min})/W)^5$*, where* t *is the current frame and the window lasts* W *frames. Writing* $I = t_{\min} + x\,(t_{\max} - t_{\min})$*,*

$$\mathbb{E}[x] \approx \Gamma\!\left(\tfrac{7}{6}\right)\left(\frac{6}{W}\right)^{1/6}.$$

Proof. The probability of surviving to fraction x of the window is

$$\prod_{k \le xW} \big(1 - (k/W)^5\big) \approx \exp\Big(-W \int_0^x u^5 \, du\Big) = \exp(-Wx^6/6).$$

Hence, using $\int_0^\infty e^{-ax^n} dx = a^{-1/n}\,\Gamma(1 + 1/n)$,

$$\mathbb{E}[x] = \int_0^1 e^{-Wx^6/6}\, dx \approx \int_0^\infty e^{-Wx^6/6}\, dx = \Big(\frac{6}{W}\Big)^{1/6} \Gamma\big(\tfrac{7}{6}\big)\,. \qquad \square$$

Example 1. At base frequency ($t_{\min} = 5\,\text{min}$, $t_{\max} = 15\,\text{min}$, 30 frames per second, $W = 18000$), $\mathbb{E}[x] \approx 0.244$ and $\mathbb{E}[I] \approx 447\,\text{s}$. With both frequency-doubling upgrades $W = 4500$ and $\mathbb{E}[x] \approx 0.308$. Taking $\pi_L = 0.42$ and $g = 1$ gives $\theta \approx 7.1 \times 10^3\,\text{s}$, about two hours, at base frequency and $\theta \approx 1.9 \times 10^3\,\text{s}$ with both upgrades ($\mathbb{E}[I] \approx 121\,\text{s}$). In the second case, with building specials on types averaging $\bar{n} = 60$ (Definition 1), $1 + \sigma_c \approx 4.5$ and $1 + \sigma_k \approx 6.1$. With three mouse upgrades (+1% of CpS per click each) at $r = 20$ clicks per second, $r\,\delta_i = 0.6\,\Delta_i$, so $\bar{\Delta}_i \approx 8.2\,\Delta_i$. A purchase is then held only if its CpS payback p_i/Δ_i exceeds about $8.2\,\theta \approx 1.6 \times 10^4\,\text{s}$, four and a half hours. Early in a run payback times are seconds to minutes, so every cookie should be spent. Late in a run payback times are hours to days, so the "Lucky!" bank should be kept.

4.2 Values under automation

Cookie Monster's simulator values upgrades by the CpS they add, so click upgrades and golden-cookie upgrades show a value of zero. Under automation they have a definite value.

Definition 1 (Automation values)**.** Let c be CpS and k the income of an auto-clicker at r clicks per second. A purchase adds Δ_i to c and $r\,\delta_i$ to k, where δ_i is its gain in cookies per click, including click power that is a share of CpS. If every golden cookie is clicked as it appears, the outcomes and their shares under the game's choice rule are: Frenzy ($\pi_F \approx 0.42$; production $\times 7$ for $77\,d\,\text{s}$), "Lucky!" ($\pi_L \approx 0.42$; $\min(0.15\,b,\ 900\,c)\,g$), building special ($\pi_B \approx 0.10$; CpS $\times(1 + n/10)$ for $30\,d\,\text{s}$, where n is the count of a random building type with at least ten), Click frenzy ($\pi_C \approx 0.04$; clicks $\times 777$ for $13\,d\,\text{s}$) and cookie storm ($\pi_S \approx 0.0085$; about $105\,d$ drops worth one to seven minutes of CpS each), with $d = 2$ under *Get lucky*. Per unit of CpS and of click income they add, on average,

$$\sigma_c = \frac{(462\,\pi_F + 3\bar{n}\,\pi_B + 25200\,g\,\pi_S)\,d}{\mathbb{E}[I]}, \qquad \sigma_k = \frac{(462\,\pi_F + 3\bar{n}\,\pi_B + 10088\,\pi_C)\,d}{\mathbb{E}[I]},$$

where $\bar{n}$ is the mean count over building types with at least ten; Frenzies and building specials multiply CpS and so also the click power derived from it. Below the cap "Lucky!" pays a share of the bank, not of production, so a purchase raises expected income by

$$\bar{\Delta}_i = (1 + \sigma_c)\,\Delta_i + (1 + \sigma_k)\,r\,\delta_i.$$

Golden cookies bring in $G = \sigma_c\,c + \sigma_k\,k + \lambda_L\,g \min(0.15\,b,\ 900\,c)$ in all, so expected income, which fills the bank, is $c + k + G$. *Lucky day* and *Serendipity*, which double the frequency, are worth G each; *Get lucky*, which doubles every duration, is worth $\sigma_c\,c + \sigma_k\,k$; and *Golden goose egg* (+5% frequency) is worth $0.05\,G$.

With income and gains measured this way, Theorem 1 and Proposition 4 apply unchanged. Both are computed without the buffs active at the moment, since a Frenzy would otherwise make every purchase look seven times better for its 77 seconds. The heavenly-upgrade values of Table 1 use the simpler shares $\sigma_F = 462\,\pi_F\,d/\mathbb{E}[I]$ and $\sigma = \sigma_F + \pi_L \min(900,\ 0.15\,b/c)\,g/\mathbb{E}[I]$, which count Frenzies and "Lucky!" only.

5 Spending heavenly chips

Chips arrive over the course of the game. Earning the chips between budget levels B and $B + dB$ takes time $g(B)\,dB/S(B)$. Here g reflects how many cookies each chip costs, which is the same whatever has been bought, and $S(B)$ is the speed. Buying heavenly upgrade j costs p_j chips and multiplies S by $m_j > 1$; the prestige factor (2) is common to all orders and can be absorbed into g.

Theorem 6 (Ratio rule for multiplicative speed-ups)**.** *Suppose there are no parent constraints and g is constant over the budget range considered. Then the order of purchases minimising the total time sorts items by*

$$\rho_j = \frac{1 - 1/m_j}{p_j}$$

in decreasing order. Any order that is not sorted can be strictly improved by swapping an adjacent pair.

Proof. Let Π be the product of the multipliers bought before two adjacent items i and j. Swapping them leaves every other term unchanged, because after both items the product is $\Pi m_i m_j$ in either order. Their own contribution is $g(p_i + p_j/m_i)/(S_0\Pi)$ with i first and $g(p_j + p_i/m_j)/(S_0\Pi)$ with j first, where S_0 is the initial speed. Hence i first is strictly better if and only if

$$p_i + \frac{p_j}{m_i} < p_j + \frac{p_i}{m_j} \iff p_i\Big(1 - \frac{1}{m_j}\Big) < p_j\Big(1 - \frac{1}{m_i}\Big) \iff \rho_i > \rho_j.$$

An unsorted order contains an adjacent inverted pair, and swapping it strictly reduces the total time. There are finitely many orders, so a sorted order is optimal. □

Remark 2. (a) For small effects $1 - 1/m \approx \ln m$, so the rule reduces to log-gain per chip. For large effects the two differ: *Heralds* can be worth $m = 2$.

(b) The rule implies *saving up*. Buying a cheaper, lower-ratio item first delays a higher-ratio one, which the interchange argument shows is worse. Cookie Monster+ therefore spends chips along the sorted order and stops at the first unaffordable item.

(c) Parent constraints are handled by treating an upgrade together with its unowned ancestors as a single block with multiplier $\prod m$ and cost $\sum p$. The interchange argument applies verbatim to adjacent blocks, which is the idea behind Sidney's decomposition for scheduling with precedence constraints (Sidney, 1975; see also Lawler, 1978). We restrict blocks to ancestor closures of single upgrades. This is exact without constraints (Theorem 6; a unit test checks it against brute force on random instances) and a heuristic in general.

Table 1 lists the multipliers used. They follow the in-game descriptions where the effect on production is direct, and are marked as estimates where the effect works through cheaper or newly unlocked store upgrades.

6 When to ascend

6.1 Time rescaling

Lemma 7 (Time rescaling)**.** *Assume A1, A3 and A4. If a run at speed* 1 *bakes $F(x)$ cookies in time x, then the same run at speed S bakes $F(S\,t)$ cookies in time t.*

Heavenly upgrade	Chips	Production multiplier m
Heavenly cookies	3	1.10
Heralds	100	$1 + 0.01 \cdot$ heralds (heralds ≤ 100)
Heavenly luck	77	$1 + 0.05\,\sigma$
Lasting fortune	777	$1 + 0.10\,\sigma_F$
Lucky digit	777	$1.01\,(1 + 0.01\,\sigma_F)$
Lucky number	77,777	$1.01\,(1 + 0.01\,\sigma_F)$
Lucky payout	77,777,777	$1.01\,(1 + 0.01\,\sigma_F)$
Kitten angels	9,000	$1 + 0.1 \cdot$ milk
Halo gloves	55,555	$1 + 0.1\,s_{\text{click}}$
Wrinkly cookies	6,666,666	1.10
Distilled essence of redoubled luck	7,777,777	$1 + 0.01\,\sigma$
Sugar baking	2×10^8	$1 + 0.01 \min(\text{unspent lumps}, 100)$
Sugar crystal cookies	10^9	1.05
Divine discount	99,999	1.01 (estimate)
Divine sales	99,999	1.005 (estimate)
Divine bakeries	399,999	1.03 (estimate)
Synergies Vol. I / Vol. II	222,222 / 2,222,222	1.05 each (estimate)
Cookie boxes, four early / three late	25 / 3.3×10^{11}	1.02 each (estimate)
Persistent memory	500	1.01 (estimate)
Starter kit / Starter kitchen	50 / 5,000	1.005 each (estimate)

Table 1: Heavenly upgrades valued by the allocation rule; s_{click} is the share of income from clicks. Chips are list prices; parents are added when blocks are formed. Cosmetics, switches, offline production, the dragon, wrinklers, seasons, fortunes and permanent upgrade slots are bought only as parents of valued upgrades.

Proof. Let $q(t)$ be the holdings and $r(q)$ the base income. At speed S the bank obeys $\dot{b} = S\,r(q)$ between purchases, and by A4 purchase decisions depend only on (b, q). In the time variable $\tilde{t} = St$ the dynamics are $db/d\tilde{t} = r(q)$ with the same decision rule, independent of S. Hence the path $(b, q)(\tilde{t})$ is the same for every S, and the cookies baked, $\int r\,d\tilde{t}$, equal $F(\tilde{t})$. □

Ranking purchases by payback period satisfies A4, since every PP_i scales by $1/S$. Golden cookies, whose spawn clock runs in real time, and the "Lucky!" reserve, which scales with S, satisfy it only approximately.

6.2 Formulation

Run k starts with all-time cookies A_k and speed $S_k = S(P(A_k))$. If it lasts T_k, it bakes $C_k = F(S_k T_k)$ (Lemma 7) and $A_{k+1} = A_k + C_k$. The player chooses the run lengths to minimise the total time $\sum_k (T_k + \tau)$ needed to reach a distant goal A^*. This is an optimal stopping problem with Bellman equation

$$V(A) = \min_{A' > A} \Big[\, T(A \to A') + \tau + V(A') \,\Big], \qquad V(A^*) = 0. \tag{4}$$

6.3 Late game: a closed form

In the late game $M(P) \approx 0.01\,h\,P$, so $S(P) = \kappa P$. Take $\tau = 0$ and assume A5. Let $y = \ln P$. A run whose log-prestige grows by Δ must bake $C = 10^{12} P^3 (e^{3\Delta} - 1)$ cookies, which at speed κP

takes

$$T(y,\Delta)=\frac{(C/\alpha)^{1/\beta}}{\kappa P}=K\,e^{\gamma y}\,\varphi(\Delta),\qquad \varphi(\Delta)=(e^{3\Delta}-1)^{1/\beta},\quad \gamma=\frac{3}{\beta}-1,\quad K=\frac{(10^{12}/\alpha)^{1/\beta}}{\kappa}. \tag{5}$$

When $\gamma>0$ ($\beta<3$), runs get longer as prestige grows; when $\gamma<0$ they get shorter.

Theorem 8 (Lower bound and optimal step)**.** *Let $G(\Delta)=(e^{\gamma\Delta}-1)/\gamma$ (with $G(\Delta)=\Delta$ if $\gamma=0$) and $w=\inf_{\Delta>0}K\varphi(\Delta)/G(\Delta)$. Every ascension schedule from y_0 to y^* satisfies*

$$\sum_k T_k\ \ge\ \frac{w}{\gamma}\big(e^{\gamma y^*}-e^{\gamma y_0}\big)\qquad \Big(\textit{resp. } w\,(y^*-y_0)\ \textit{if } \gamma=0\Big),$$

with equality if and only if every step Δ_k minimises φ/G. For $\beta>1$ a minimiser $\Delta^\star$ exists.

Proof. For $\gamma\neq 0$, $G(\Delta)>0$ for $\Delta>0$, and $e^{\gamma y}G(\Delta)=(e^{\gamma(y+\Delta)}-e^{\gamma y})/\gamma$. By definition of w, $K\varphi(\Delta)\ge w\,G(\Delta)$ for every $\Delta>0$. With $y_{k+1}=y_k+\Delta_k$,

$$\sum_k T_k=\sum_k Ke^{\gamma y_k}\varphi(\Delta_k)\ge w\sum_k e^{\gamma y_k}G(\Delta_k)=\frac{w}{\gamma}\sum_k\big(e^{\gamma y_{k+1}}-e^{\gamma y_k}\big)=\frac{w}{\gamma}\big(e^{\gamma y^*}-e^{\gamma y_0}\big),$$

a telescoping sum. The only inequality used is an equality exactly when $K\varphi(\Delta_k)=wG(\Delta_k)$ for every k. The case $\gamma=0$ is identical with $G(\Delta)=\Delta$.

For existence: as $\Delta\to 0^+$, $\varphi(\Delta)/G(\Delta)\sim 3^{1/\beta}\Delta^{1/\beta-1}\to\infty$ because $\beta>1$. As $\Delta\to\infty$, $\varphi(\Delta)\sim e^{3\Delta/\beta}$ while G grows at most like $e^{\gamma\Delta}/\gamma$ with $\gamma<3/\beta$, so the ratio again tends to infinity. A continuous function on $(0,\infty)$ that tends to infinity at both ends attains its minimum. □

The bound is achieved by repeating the step $\Delta^\star$, up to the rounding of the last step before the goal. For $\beta\le 1$ the ratio φ/G has no interior minimum and resets should be as frequent as overhead allows. This degenerate case motivates a safeguard in the implementation (Section 6.5).

Corollary 9 (Stopping rule)**.** *At an interior minimiser $\varphi'/\varphi=G'/G$. Since the current run's length is proportional to $\varphi(\Delta)$, the optimal moment to ascend is when the run's marginal log-prestige rate falls to*

$$\frac{d\Delta}{dT}=\frac{1-e^{-\gamma\Delta}}{\gamma\,T}.$$

For a run whose cookies grow like T^β, $\Delta=\frac{1}{3}\ln v$ with $v=(A+C)/A$ and $d\Delta/dT=\frac{\beta}{3}(1-1/v)/T$. Ascending is optimal when all-time cookies have grown by the factor $v^\star$ solving

$$\frac{\beta}{3}\Big(1-\frac{1}{v}\Big)=\frac{1-v^{-\gamma/3}}{\gamma}. \tag{6}$$

Proof. $T'(\Delta)/T(\Delta)=\varphi'/\varphi=G'/G$, hence $d\Delta/dT=G/(G'\,T)=(e^{\gamma\Delta}-1)/(\gamma e^{\gamma\Delta}T)$. With $C\propto T^\beta$ we have $dv/dT=\beta(v-1)/T$, so $d\Delta/dT=\frac{1}{3v}\,dv/dT$. Equating the two expressions gives (6). □

Remark 3 (The marginal value theorem). For $\gamma=0$ ($\beta=3$) the rule reads $d\Delta/dT=\Delta/T$: ascend when the marginal rate equals the average rate. This is the marginal value theorem of optimal foraging (Charnov, 1976). Applying it when $\beta\neq 3$ ignores that later runs get longer or shorter. It solves $\beta(v-1)=v\ln v$ instead of (6) and errs by a factor of 1.6 at $\beta=2$ and 6.5 at $\beta=6$ (Table 2).

Remark 4 (Overhead). With $\tau>0$ the problem is no longer self-similar. Corollary 9 then holds approximately with T replaced by $T+\tau$; the general problem is solved exactly by dynamic programming (Section 6.5).

β	γ	$v^\star$ (eq. 6)	prestige × per run	naive MVT v	brute force	dynamic program
1.5	1.000	4.24	1.62	2.40	4.42	4.39
2.0	0.500	7.87	1.99	4.92	7.95	7.18
2.5	0.200	12.03	2.29	9.31	11.49	11.50
3.0	0.000	16.80	2.56	16.80	15.95	16.69
4.0	−0.250	28.39	3.05	50.44	31.69	28.95
6.0	−0.500	60.91	3.93	397.38	63.59	60.13

Table 2: Optimal factor $v^\star$ by which each run multiplies all-time cookies in the late game. "Brute force" scans fixed factors to grow prestige a millionfold (the last, partial run causes the scatter). "Dynamic program" is the planner of Section 6.5, averaged over runs that have baked 0.1, 0.5 and 2 times the previous all-time cookies.

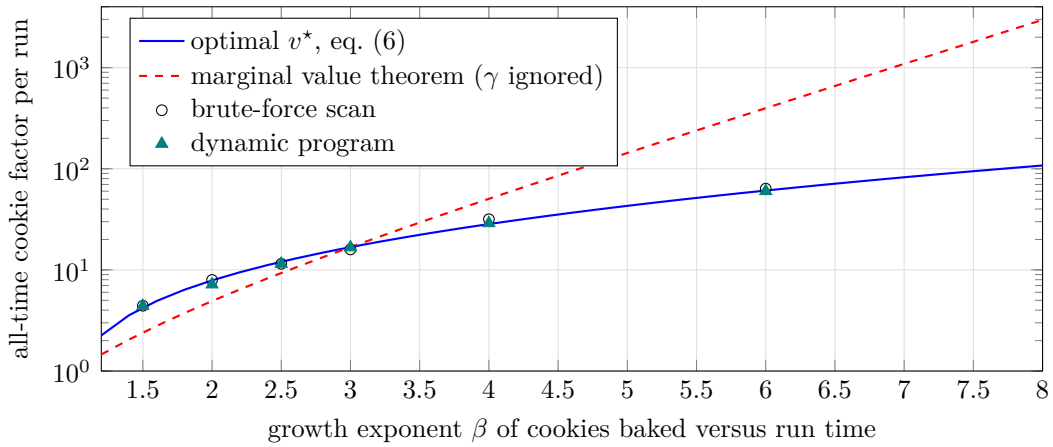


Figure 1: Late-game optimum versus the uncorrected marginal value theorem. The two agree only at $\beta = 3$.

6.4 Early game: why early resets fail

Proposition 10 (No prestige effect, no reset)**.** *If the speed is the same in every run ($S(P) \equiv S$) and $F(x) = \alpha x^\beta$ with $\beta > 1$, then for any $\tau \geq 0$ the fastest way to bake a given number of cookies is a single run.*

Proof. Splitting D cookies into runs of $D_1, \ldots, D_n$ costs $\sum_k (D_k/\alpha)^{1/\beta}/S + (n-1)\tau$. Since $q = 1/\beta \in (0,1)$, the map $x \mapsto x^q$ is strictly subadditive on $[0,\infty)$: $(\sum_k D_k)^q < \sum_k D_k^q$ for $n \geq 2$ positive parts. Hence one run is strictly fastest. □

Early in the game, ascending raises the speed only slightly, so the problem is close to this regime and premature resets are costly. An earlier version of the advisor applied the late-game rule, with a numerical clamp on γ, to a first run of 4.89×10^{12} cookies (prestige 1.70, one level, +1% production) and recommended ascending. The planner below says to keep baking and puts the cost of ascending at that point at 11–16 hours of total progress (Table 3).

6.5 The general planner

Speed model. A run starting at prestige P has speed

$$S(P) = \Big(\prod_{j \in J(\lfloor P \rfloor)} m_j \Big) \big(1 + 0.01\, h\, \lfloor P \rfloor\big),$$

where Legacy is bought with the first chip and $J(\lfloor P \rfloor)$ is the longest prefix of the order from Theorem 6 that the chips available at P can pay for, with parents included.

Run-time model. The current run has baked C cookies in time T, and $\beta = cT/C$. A fresh run rebuilds to C convexly, then grows like the current run:

$$t(X) = \begin{cases} T\,(X/C)^{1/\beta_r}, & X \le C, \quad \beta_r = \max(\beta, 1.5), \\ T\,(X/C)^{1/\beta}, & X > C, \end{cases}$$

and a run at speed S needs $t(X)\, S_{\text{now}}/S$ to bake X cookies (Lemma 7). The floor $\beta_r \ge 1.5$ encodes that every run starts from zero production, even when the current run has stalled.

Proposition 11 (Exactness of the dynamic program)**.** *Restrict ascensions to a finite grid of all-time cookie levels $a_0 < a_1 < \cdots < a_n = A^*$, and let $t(i, j)$ be the duration of a run from a_i to a_j. Then*

$$V(i) = \min_{j > i} \Big[t(i, j) + \tau\, \mathbf{1}[j < n] + V(j) \Big], \qquad V(n) = 0,$$

computed for $i = n-1, \ldots, 0$ in $O(n^2)$ time, is the minimum total time from a_i, and the minimising j is the optimal next ascension point. For the current run, the optimal target minimises $\big(t(a_j - A) - T\big) + \tau + V(j)$.

Proof. All-time cookies only increase, so schedules are paths in a directed acyclic graph on the grid with edge weights $t(i, j) + \tau$. The recursion is Bellman's equation (4) on this graph. By induction on $n - i$, $V(i)$ equals the shortest path length from i to n. □

Safeguards. Four implementation choices keep the planner robust.

- *Horizon.* The goal is the level the current growth would reach in 10^4 run lengths. Without this, durations near a very distant goal swamp the hour-scale differences of the current decision in floating-point arithmetic.
- *End effects.* The target is the median over five goals spaced over one decade of all-time cookies, which cancels the dependence on how the last run fits before the goal.
- *Minimum gain.* An ascension must raise speed by at least 10%. This guards against a stalled run (for example while nothing is bought), which pushes β towards 1 where Theorem 8 favours constant resets.
- *Grid and age.* The grid has 400 log-spaced levels plus every whole prestige level below 400, and no plan is made before a run is ten minutes old.

β	ascend now?	optimal target	forecast (growth ±25%) / cost of ascending now
1.5	no	prestige 4 (+14.4% speed)	50.6–129.4 h / 11.4 h
2.0	no	prestige 4 (+14.4% speed)	30.9–68.6 h / 14.2 h
2.5	no	prestige 4 (+14.4% speed)	22.0–45.0 h / 15.2 h
3.0	no	prestige 4 (+14.4% speed)	16.9–33.0 h / 15.8 h
4.0	no	prestige 4 (+14.4% speed)	11.6–21.2 h / 16.3 h

Table 3: The planner on a first run of 4.89×10^{12} cookies in 17.2 hours without Legacy. Prestige 4 pays for Legacy and *Heavenly cookies* together, so the first ascension buys several heavenly upgrades at once.

7 The rest of the game

The policies above settle purchases, golden cookies and ascension. The game has further systems, most of them reset at every ascension. Each is handled by a rule judged with the same long-run income model as purchases (Definition 1), extended below to wrath cookies and wrinklers. This section states the facts the rules rest on.

7.1 Achievements are worth production

Proposition 12 (Value of an achievement)**.** *Each achievement (shadow ones aside) raises milk μ by 0.04. With kitten upgrades of factors k_i and a milk multiplier m, CpS carries the factor $\prod_i (1 + k_i m \mu)$, so one more achievement multiplies CpS by*

$$\prod_i \frac{1 + k_i m(\mu + 0.04)}{1 + k_i m \mu} \;\approx\; \exp\Bigl(0.04 \sum_i \frac{k_i m}{1 + k_i m \mu}\Bigr).$$

Proof. The game multiplies CpS by $1 + k_i m \mu$ for each kitten owned; substitute $\mu + 0.04$ and take the ratio. The approximation is $\ln(1 + x) \approx x$ for each factor. □

With the eight kittens of factors 0.1, 0.125, 0.15, 0.175, 0.2, 0.2, 0.2, 0.1 and $\mu = 7.88$ (an observed save), one achievement adds 2.2% of CpS, and making milk 10% stronger (Mokalsium in the first Pantheon slot) multiplies CpS by 1.52. Achievements that only take an action (clicking the news ticker 50 times, the tiny cookie, the madeleine, selling one grandma, a golden cookie clicked in its last second, a wrinkler poked 50 times, pressing one of Santa's helper grandmas, clicking the ticker in a window shrunk below 900 pixels for a moment, and a gift of one cookie sent to oneself and redeemed an hour later) are therefore taken at once. Actions that cost something are priced in time:

Proposition 13 (What an achievement is worth in time)**.** *Suppose, as in Lemma 7, that every future run retraces its trajectory at a speed proportional to income. A permanent gain of a share x of income from now on is then worth $xH/(1+x) \approx xH$ seconds of progress over the remaining play time H, while a pause of D seconds costs D, and so do C cookies spent at income I ($D = C/I$). An action that earns achievements worth $x_1, \dots, x_k$ (Proposition 12) at a delay D pays if and only if $D < H \sum_i x_i$.*

Proof. With every production rate multiplied by $1 + x$, the state reached at wall time t is the one the unboosted game reaches at $(1 + x)t$, so at the horizon the boosted game is xH seconds ahead; equivalently, the remaining time to any fixed goal shrinks from H to $H/(1 + x)$. A pause shifts the whole trajectory by D. □

The horizon is unknown. With no other information (a scale-free prior on the total length of play), the median remaining time equals the time played so far, so autoplay takes H to be the age of the save. On the observed save ($x = 2.14\%$ after 77 hours) one achievement is worth 1.66 hours. This prices three kinds of action.

Born again. Neverclick (a million cookies with at most 15 clicks) and Hardcore (a billion without an upgrade) count only in a Born again run, which starts from nothing: no prestige, heavenly upgrades, sugar lumps or minigames. A simulation of such a run (buildings only, the 15 clicks spent on the first cursor, golden cookies clicked) reaches a million cookies in 41 minutes and a billion in 3.8 hours. One run earns both, so autoplay starts one at a reincarnation once $2xH$ exceeds four hours, at about 93 hours of play on that save. It then clicks the big cookie exactly 15 times, buys only buildings, and ascends back into a normal run as soon as both are won.

Bursting sprees. Wrinklesquisher and Moistburster need 50 and 200 wrinklers burst within one run. A spree bursts each wrinkler as it appears. Every slot refills independently after τ seconds on average (222 s at stage 3 with Unholy bait), so N bursts take $N\tau/n_{\max}$: 15 and 62 minutes with twelve slots, during which income falls from that of the better stage to that of wrath without wrinklers. Autoplay bursts when the loss is worth less than xH and the planner expects the run to last long enough.

Spell counts. Casting the cheapest spell instead of Force the Hand of Fate roughly doubles the number of casts (magic regenerates like $\sqrt{m}$, Proposition 18), reaching 99 casts in about 22 hours instead of 41. That brings two achievements forward by hours, worth about half an hour, but gives up golden cookies worth about an hour and a half, so autoplay keeps casting the Hand of Fate.

7.2 Godzamok

Godzamok, one of the Pantheon's gods, rewards selling buildings: each building sold adds $b = 1\%$ (0.5% in the second slot, 0.25% in the third) to the click power of a ten-second "Devastation" buff, and sales while the buff runs add to it without limit and without extending it.

Proposition 14 (Godzamok round trips)**.** *Selling all n copies of a building whose next copy costs p and buying them back in the same frame leaves production unchanged and costs*

$$\big(1 - 1.15\,\sigma\big)\,\frac{p\,(1 - 1.15^{-n})}{0.15} \qquad (\sigma = 0.25,\ \textit{the sale refund}),$$

0.7125 *times the cost of buying them back. It adds bn to the buff's multiplier, worth $b\,n\,K\,t$ cookies, where K is the click income without the buff and t the buff time left. If U buildings are cycled in each of the $F = 300$ frames of a buff, the click multiplier averages $1 + bU(F+1)/2 \approx 1 + 150\,bU$.*

Proof. A sale at amount j refunds σ times the price of copy $j+1$, which is 1.15 times the price of copy j; summed over the top n copies, the refund is $1.15\,\sigma$ times the buy-back cost, a geometric sum. Click power is linear in the buff's multiplier, so a sale adds bnK per second for the remaining t seconds. Cycling U buildings per frame makes the multiplier $1 + bUk$ in frame k; average over $k = 1, \dots, F$. □

Autoplay cycles, every frame and within three milliseconds of it, each building type whose round trip returns at least twice its cost, except grandmas (selling the last one ends the Elder Pledge). Late in a run that is every type but the newest tiers, whose copies cost more than they add. The mouse upgrades make a click worth 15% of CpS, so at 20 clicks a second $K \approx 3c$ before the buff. On the observed save, cycling about 2,200 buildings a frame kept the multiplier at 3,329 on average and income at about 22,000 times CpS, against a few times CpS before; the game kept 30 frames a

Stage	Frenzy	"Lucky!"	Special	Click fr.	Storm	Chain	Elder fr.	Clot	Ruin	Cursed
0	.422	.422	.100	.040	.008	.008	–	–	–	–
1	.294	.341	.076	.032	.024	.023	.017	.090	.090	.008
2	.157	.280	.056	.027	.039	.039	.034	.171	.171	.015
3	–	.245	.041	.023	.055	.055	.051	.245	.244	.023

Table 4: Golden-cookie outcome shares by Grandmapocalypse stage (at least ten buildings, no dragon auras; building debuffs, 1.8% at stage 3, omitted).

second. The economy model includes the average multiplier in click income, so the Pantheon slots Godzamok as soon as a worship swap allows. Each ascension restores three swaps; once they are spent the next one takes 16 hours, so when a much better arrangement waits for swaps, autoplay keeps the next sugar lump for the refill that restores all three.

7.3 Wrinklers and wrath cookies

Proposition 15 (Wrinkler factor)**.** *With n wrinklers digesting, a share $s = \min(1, n/20)$ of CpS c is withered, and every wrinkler accumulates sc per second. Popped, a wrinkler returns $r \geq 1.1$ times its contents. Income from CpS is therefore $c\,W(n)$ with*

$$W(n) = 1 - s + s\,n\,r, \qquad W(10) = 6, \quad W(12) = 8.32 \quad (r = 1.1).$$

Proof. In the game's update each digesting wrinkler adds cs per second to its stomach, the whole withered share rather than $1/n$ of it, while the bank receives $(1-s)c$. Summing over the n wrinklers and applying the return factor on popping gives the claim. □

Wrinklers only appear during the Grandmapocalypse, whose stage $\ell \in \{1, 2, 3\}$ turns each golden cookie into a wrath cookie with probability $\ell/3$. Wrath cookies trade Frenzies for clots (production halved for 66 s) and ruin (losing $\min(0.05\,b, 600\,c)$), but they add Elder frenzies (production $\times 666$ for 6 s), cursed fingers (each click worth 10 s of production for 10 s) and, above all, cookie storms: about 105 golden cookies in 7 s, each worth one to seven minutes of CpS, all collected by automation. Table 4 gives the outcome shares, simulated from the game's choice rule.

Proposition 16 (Wrath cookies win under automation)**.** *Measure each outcome by the extra seconds of CpS it brings, buffs counting W times (wrinklers digest buffed CpS) and payouts once. With click income $k = 0.6\,c$, a full* "Lucky!" *bank and building specials worth 588 s, an average golden cookie is worth 1248 s of CpS at stage 0 and 2141 s at stage 3 without wrinklers. With ten wrinklers the stage-3 cookie is worth 3236 s, and total income is about* 2.8 *times that of stage 0.*

Proof. Multiply the shares of Table 4 by the effect sizes: Frenzy $462(Wc + k)$, building special $3\bar{n}(Wc + k)$, Elder frenzy $3990(Wc + k)$, clot $-33(Wc + k)$, Click frenzy $10088\,k$, storm $25200\,c$, cursed finger $2000\,c - 10(Wc + k)$, "Lucky!" $900\,c$, chain $1290\,c$, ruin $-300\,c$, and sum. □

Autoplay therefore researches up to the Elder Pact, buying the research that the game guards with a confirmation prompt. Godzamok changes the verdict:

Proposition 17 (Calm wins once clicks dominate)**.** *With the Elder Pact, a pledge costs at most $8^{14} \approx 4.4 \times 10^{12}$ cookies and comes back by itself when it runs out, so the stage is a free choice between calm (stage 0, no wrinklers) and wrath (stage 3, $W(n_{\max})$). With effect duration $d = 2$, a mean of 200 copies per building type, $\mathbb{E}[I] = 121$ s and twelve wrinklers, golden cookies multiply*

click income by 11.8 *in calm and* 8.5 *in wrath, and CpS income by* 8.7 *and* 62.8*. Calm earns more if and only if click income exceeds about 16 times CpS.*

Proof. For clicks, Frenzies, building specials and Click frenzies bring $595\,d + 0.30\,\bar{n}\,d$ buff-seconds per golden cookie in calm; in wrath, Elder frenzies and Click frenzies, less clots, cursed fingers and debuffs, bring $428\,d + 0.12\,\bar{n}\,d$ (shares from Table 4, effect sizes as in Proposition 16). Divide by $\mathbb{E}[I]$ and add one. For CpS the same buffs count W times, and storms and cursed fingers pay directly. The threshold is $(62.8 - 8.7)/(11.8 - 8.5) \approx 16$. □

Without Godzamok clicks bring about $3c$ and wrath wins; with him they bring thousands of times c, and calm earns about 40% more. Autoplay compares the two incomes with the economy model and renews the pledge whenever calm wins. It still lets the wrath reach stage 3 once (the Grandmapocalypse achievement), runs the elder achievements below, and then revokes the covenant they leave, which costs 5% of CpS while a pledge costs nothing. The bank's marginal yield changes with the stage too, because ruin takes 5% of the bank: per banked cookie it is $(0.15\,\pi_L g + 0.215\,\pi_{\text{chain}} g - 0.05\,\pi_{\text{ruin}})/\mathbb{E}[I]$, so the threshold θ of Proposition 4 rises from about 1.9×10^3 s at stage 0 to 3.3×10^3 s at stage 3.

When to pop. Wrinklers lose nothing by waiting, but their contents earn nothing until popped. Popping a wrinkler holding X cookies now rather than at the ascension, T seconds away, adds $X(e^{\rho T} - 1)$ by the end, where $\rho = (\beta - 1)/t$ is the run's income growth rate. Its slot then stays empty for the respawn time τ, during which the other wrinklers each digest a twentieth less: the loss is $L = \big(0.05\,(2n-1)\,r - 0.05\big)c\,\tau$, compounding to $Le^{\rho T}$. Autoplay pops the fattest normal wrinkler when $X > Le^{\rho T}/(e^{\rho T} - 1)$.

Elder achievements. With the Elder Pact owned, the pledge returns as soon as a signed covenant is revoked, so pledge, covenant and revoke repeated five times earn all three elder achievements in seconds, for a negligible amount of cookies, instead of two and a half hours of peace.

7.4 Minigames

Proposition 18 (Cast at full magic)**.** *Below its cap M, magic regenerates at rate $\kappa\sqrt{m}$. A spell costing $C \le M$ is cast most often by casting whenever magic reaches M.*

Proof. Refilling from a to $a + C$ takes $\int_a^{a+C} dm/(\kappa\sqrt{m}) = (2/\kappa)(\sqrt{a+C} - \sqrt{a})$, which decreases in a. Every cycle of a periodic policy refills some interval of length C, and $a = M - C$ is the largest possible start. □

With $M \approx 72$ and Force the Hand of Fate costing $C \approx 53$, casting at full yields one cast every 23 minutes instead of every 40.

Pantheon. Three slots take three of eleven gods. Autoplay evaluates every ordered triple of the useful ones with the game's CpS formula and the income model. Mokalsium's and Jeremy's golden-cookie penalties enter through the interval. It moves gods only when income rises by more than 1% and worship swaps are available.

Garden. New seeds come only from mutations, which grow in empty tiles next to mature plants of the right kinds.

Lemma 19 (Checkerboard breeding)**.** *On a rectangular plot of at least 2×2 tiles, plant parent kind A on the tiles with $x + y$ even and x even, kind B on those with $x + y$ even and x odd, and*

leave the tiles with $x+y$ odd empty. Then every empty tile touches both kinds (two parents of kind A when $A = B$), so all of them can mutate into a seed bred from A and B.

Proof. An empty tile (x, y) has $x+y$ odd. Its horizontal neighbours $(x \pm 1, y)$ are parent tiles with the parity of $x+1$, its vertical neighbours $(x, y \pm 1)$ parent tiles with the parity of x. In a plot at least two tiles wide and tall, at least one of each pair lies inside the plot. □

Autoplay breeds along the chains whose parents can be planted (Thumbcorn from wheat, then Cronerice, Gildmillet, Clover, Shimmerlily, Elderwort, Golden clover, Bakeberry and onward), uproots whatever else sprouts in the empty tiles, lets each new kind grow up and picks it when mature, which unlocks its seed. Wood chips (three mutation rolls per tick, from 300 farms) or else fertilizer (a tick every three minutes instead of five) speed this up. Thumbcorn (+2% click power) is worth 2% of nearly all income once Godzamok multiplies clicks, and Golden clover (+3% golden-cookie frequency) 3% of the golden-cookie income. The plot is cleared at every ascension, so breeding advances in runs long enough for the parents to mature. With nothing left to breed, a seed j with effect v_j on long-run income, mature lifetime T_j and price p_j is planted, on clay (effects $\times 1.25$), when $v_j T_j > p_j$ and p_j / v_j beats the payback of the auto-buyer's next purchase. Plants are harvested while mature, just before the growth tick that could kill them, which counts toward the harvest achievements. One tile stays empty, since mutations only grow in empty tiles.

Stock Market. Prices revert toward the resting value $R_i = 10 + 10\,i + (\ell_{\text{bank}} - 1)$. Buying carries an overhead of at most 20% and selling none, so buying below $0.6\,R_i$ and selling at R_i returns at least $1/(0.6 \cdot 1.2) - 1 = 39\%$ per completed round trip. The rule uses only the prices the market shows.

Offices and loans. Office upgrades sacrifice 100 to 700 cursors and need cursor levels 2, 4, 8, 10 and 12; they open loans. Loan i multiplies CpS by m_i for t_i and then by $m'_i < 1$ for t'_i, against a down payment of a share f_i of the bank: $(m, t, m', t', f) = (1.5, 2\,\text{h}, 0.25, 4\,\text{h}, 20\%)$, $(2, 40\,\text{s}, 0.1, 40\,\text{min}, 40\%)$ and $(1.2, 48\,\text{h}, 0.8, 120\,\text{h}, 50\%)$. A run that ends before t_i never pays the interest, and ascending then also earns Debt evasion. Autoplay takes a loan once the planner's slow estimate of the time left is at most $0.9\,t_i$. It ends the run early when a loan is about to run out and ascending now costs less than the interest, $(1 - m'_i)\min(t'_i, \text{time left})$ seconds, and it takes any open loan right before ascending, when the bank is about to be lost anyway.

7.5 The dragon, seasons, sugar lumps and permanent slots

Dragon. Sacrificing n copies of a building priced p costs $p\,(1 - 1.15^{-n})/0.15 \approx 6.67\,p$ to buy back ($n = 100$). The dragon starts over every run. Training toward Radiant Appetite (production $\times 2$, the sacrifice of 100 copies of each of the first 14 buildings) goes ahead once all those buildings are at hand and buying them back costs at most an hour of income. The same holds for Dragon Guts (two more wrinklers that digest and return 20% more, W up to about 11.7). Auras are chosen by evaluation, and switching costs one top building.

Seasons. Christmas brings Santa's 14 levels, costing $\sum_{j=1}^{14} j^j \approx 1.2 \times 10^{16}$ cookies, with a gift at each level (Santa's legacy gives +3% per level). It also brings reindeer worth one to two minutes of CpS each. Valentine's day brings seven heart biscuits (+2% each), and Easter brings eggs. Each run visits Christmas until Santa is complete and Valentine's day once the hearts cost a minute of production, then Easter until seven eggs while that achievement is missing, and otherwise stays in Christmas. The auto-buyer keeps up to a minute of production for the next seasonal action.

Sugar lumps. Lumps go first to the minigames, then to the Sugar frenzy each run once Sugar craving is owned (production $\times 3$ for an hour for one lump, two hours of income), then to cursor levels until the first loan opens (11 lumps counting the bank's own level, for +50% CpS in the last

Speed, target	play	payback period	exchange index	with look-ahead
$S = 1$, 10^{15}	continuous	68.73 h	0.00%	+0.03%
	every 10 min	70.33 h	−0.17%	−0.09%
	every 60 min	77.39 h	−0.06%	−0.08%
$S = 20$, 10^{20}	continuous	36.58 h	0.00%	+0.09%
	every 10 min	38.62 h	−0.02%	+0.01%
	every 60 min	46.65 h	−0.97%	−1.68%
$S = 200$, 10^{25}	continuous	627.81 h	0.00%	+0.35%
	every 10 min	629.95 h	0.00%	+0.35%
	every 60 min	638.06 h	−0.12%	+0.19%

Table 5: Time to bake a target number of cookies. The last two columns give the speed-up relative to the payback period (negative means slower).

two hours of every run), and then to building levels. The gamble on a mature but unripe lump (Hand-picked; half the time it yields nothing) is taken once the minigames are open. A level adds 1% of building b's CpS, a share $0.01\, s_b/(1 + 0.01\, \ell_b)$ of CpS, for $\ell_b + 1$ lumps, plus Proposition 12's value at level 10. Levels are bought by gain per lump, saving up for the best as in Theorem 6's remark.

Permanent slots. Milk carries over, so a kitten kept in a slot multiplies CpS by $1+km\mu$ from the first second of each run, and a cookie upgrade by 1+ its power. The slots take last run's upgrades in decreasing order of the logarithm of that multiplier. These systems also enter the heavenly-upgrade values (estimates): How to bake your dragon 1.8, Season switcher 1.3, permanent slots 1.12 down to 1.03, Elder spice 1.25, Unholy bait 1.05, Sacrilegious corruption 1.04 and Wrapping paper 1.02 (the self-sent gift's achievement).

8 Numerical validation

Simulator. The simulator uses the game's formulas for the base price and production of all 21 buildings, tiered upgrades that double a building at 1, 5, 25, 50, 100, … copies, global +10% upgrades whose prices grow twentyfold, a milk proxy $1 + n/K$ for n buildings, and clicking at 5 cookies per second. Purchases are greedy by the exchange index. Ascension rules are compared by the prestige reached after a fixed play time, as a percentage of the best rule for that horizon, averaged over horizons of 60 to 960 days.

Purchase order. Table 5 compares the payback period with the bank-independent index of Theorem 1 and with a look-ahead that values a building together with the tier upgrade it unlocks. Differences are negligible in continuous play. In infrequent play, where large banks build up, the payback period is slightly better.

Ascension timing. Table 6 compares the dynamic-programming planner, the stopping rule of Corollary 9 with β measured live, the uncorrected marginal value theorem, and fixed rules that ascend when all-time cookies have grown v-fold (v from 1.5 to 500).

Both the planner and the stopping rule beat the uncorrected marginal value theorem in every scenario and the hindsight-tuned fixed rule on average. In these late-game scenarios the simpler stopping rule is slightly better (mean 98.6% against 97.6%). The gap comes from extrapolation. The planner projects the current growth exponent forward, but simulated runs keep decelerating,

Scenario	planner (DP)	stopping rule	naive MVT	best fixed v	worst fixed v
$K = 2500$, $A_0 = 10^{15}$	93.5	98.2	93.9	79.3 ($v = 60$)	13.4 ($v = 1.5$)
$K = 2500$, $A_0 = 10^{24}$	98.5	98.7	93.9	98.7 ($v = 8$)	79.1 ($v = 1.5$)
$K = 500$, $A_0 = 10^{15}$	98.8	98.0	90.8	84.5 ($v = 12$)	20.5 ($v = 1.5$)
$K = 500$, $A_0 = 10^{24}$	98.2	98.7	94.0	97.9 ($v = 12$)	68.2 ($v = 500$)
$K = 100$, $A_0 = 10^{15}$	98.1	98.9	91.1	92.8 ($v = 12$)	34.1 ($v = 1.5$)
$K = 100$, $A_0 = 10^{24}$	98.3	99.3	91.6	98.9 ($v = 8$)	45.6 ($v = 500$)
Mean	97.6	98.6	92.6	92.0	–

Table 6: Prestige reached as a percentage of the best rule at each horizon, averaged over five horizons (60–960 days). K is the milk parameter; A_0 the all-time cookies at the start. "Stopping rule" is Corollary 9 with β measured live. The "best fixed" rule is chosen in hindsight per scenario; no single v is best everywhere.

Rule	30 days	120 days	ascensions (prestige levels, 120 days)
Dynamic-programming planner	**658.3**	**18,512.5**	4, 206, 815, 1759, 3789, 7321, 13092, 18512
Fixed $P_{\min} = 150$, $v = 20$ (best fixed)	421.2	17,939.4	150, 407, 1105, 3000, 8143, 17939
Fixed $P_{\min} = 150$, $v = 8$	359.7	17,686.5	150, 300, 600, …, 17686
Fixed $P_{\min} = 50$, $v = 20$	368.8	17,137.6	50, 135, 368, 1000, …
Fixed $P_{\min} = 4$, $v = 3$	68.1	6,812.0	4, 5, 8, 12, 17, …

Table 7: Final prestige from a fresh game (12 fixed rules tested; the best and three others shown).

so continuing looks slightly cheaper than it is: in the first scenario the planner's first two runs last 1033 and 917 hours against the stopping rule's 879 and 833, which costs about five points. A run-curve model that anticipates deceleration is the natural fix. The stopping rule, however, assumes $S \propto P$; the planner is the policy that remains valid from a fresh game.

From a fresh game. Table 7 starts with no prestige, so Legacy, *Heavenly cookies* and prestige power all enter through the speed model. Fixed rules ascend once prestige is at least $P_{\min}$ and all-time cookies have grown v-fold.

9 Implementation

The policies run in Cookie Monster+, a fork of the Cookie Monster mod for the Steam version of the game (2.053). Every second the mod records the ascension inputs. Every five seconds it re-solves the dynamic program, which takes 10–60 ms; in testing no main-thread task exceeded 50 ms. With automation enabled, the mod also:

- clicks the big cookie 20 times per second (the game ignores clicks less than 20 ms apart);
- clicks every golden cookie and reindeer as soon as it appears;
- every frame with Godzamok slotted, sells and buys back the building types of Proposition 14 within three milliseconds; the counts are restored before the frame ends, so only click power is recomputed;
- every nine frames, buys the item with the smallest payback period, computed with the automation values of Section 4 and the "Lucky!" reserve of Proposition 4. It skips switches, sugar-lump costs and the Chocolate egg;

- on the frames in between, runs the systems of Section 7: the Grandmapocalypse (stage choice, sprees and wrinklers), the dragon, seasons, sugar lumps, the Pantheon, Grimoire, Garden and Stock Market, and achievements that only take an action;
- once the planner has said "ascend now" for ten seconds, pops normal wrinklers, buys the Chocolate egg, ascends, spends chips along the order of Theorem 6 (parents first, saving up for the next item), fills the permanent slots and reincarnates, into a Born again run when Proposition 13 says the challenge pays (or early, when a loan is about to reach its interest).

The mathematical components are covered by 124 unit tests. These include reproduction of Table 2's closed form by the dynamic program, refusal to ascend in the first-run case, and brute-force optimality of the chip order.

10 Limitations

- The run curve is extrapolated from one number, the current growth exponent. Real runs accelerate when new tiers of upgrades unlock and slow down when they run out. In simulation, extrapolating a decelerating run made the planner too patient by about 15% in its first runs (Section 8).
- Heavenly multipliers marked as estimates in Table 1 are not derived from the game's code paths, and blocks restricted to ancestor closures are a heuristic for general precedence graphs.
- Golden-cookie probabilities π_F, π_L are approximate, and Assumption A4 holds only approximately for golden cookies and the "Lucky!" reserve.
- The analysis is risk-neutral. The rules of Section 7 rest on the income model and on simulated outcome shares; several heavenly values, the Garden's seed values and the Stock Market thresholds are estimates. Garden breeding covers the chains whose parents can be planted (fungi only once weeds leave their spores), and Skruuia, the Dragonflight aura, brokers and spell counts are not pursued. The play horizon of Proposition 13 is a prior, not a measurement.
- The simulator is a stylised economy. Its agreement with the theory supports the internal consistency of the policies, not their exact optimality in the real game.

References

[1] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[2] E. L. Charnov. Optimal foraging, the marginal value theorem. *Theoretical Population Biology*, 9(2):129–136, 1976.

[3] E. L. Lawler. Sequencing jobs to minimize total weighted completion time subject to precedence constraints. *Annals of Discrete Mathematics*, 2:75–90, 1978.

[4] J. B. Sidney. Decomposition algorithms for single-machine sequencing with precedence relations and deferral costs. *Operations Research*, 23(2):283–298, 1975.

[5] W. E. Smith. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3(1–2):59–66, 1956.

[6] J. Thiennot (Orteil). *Cookie Clicker*, version 2.053. DashNet, 2013–2025.

[7] Cookie Monster Team. *Cookie Monster*, a Cookie Clicker add-on. `https://github.com/CookieMonsterTeam/CookieMonster`.

A Algorithms

Algorithm 1 Ascension planner (every five seconds)

measure A, C, run time T, income c; set $\beta \leftarrow \min(\max(cT/C, 0.3), 12)$
build the chip order of Theorem 6; define $S(P)$ as in Section 6.5
for each of five goals a_k^* spaced over one decade beyond the 10^4-run-length horizon **do**
    grid $\leftarrow$ 400 log-spaced levels in $[A + C, a_k^*] \cup$ whole prestige levels < 400
    solve V backwards (Proposition 11); choose target j_k minimising remaining$(j) + \tau + V(j)$
    among targets with speed gain $\geq 10\%$ only
end for
target $\leftarrow$ median of $j_1, \ldots, j_5$; **ascend now** iff target is the current level

Algorithm 2 Automation tick (every frame; clicks on a separate 50 ms timer)

click every golden cookie and reindeer on screen
if frame mod$6 = 0$ and the store is in buy mode **then**
    for each candidate i: value $\bar{\Delta}_i$ (Definition 1), reserve $R_i \leftarrow$ "Lucky!" bank if $p_i/\bar{\Delta}_i > \theta$, else 0
    $i^* \leftarrow \arg\min_i \max(p_i - (b - R_i), 0)/(c + k + G) + p_i/\bar{\Delta}_i$
    buy i^* if $p_{i^*} \leq b - R_{i^*}$
end if
if the planner has said "ascend now" for 10 s **then**
    pop normal wrinklers; buy the Chocolate egg; ascend; spend chips along the order; reincarnate
end if